

Walmart SQL Commands

1. Test That All 8 Tables Exist

```
SELECT table_name
FROM user_tables
WHERE table_name IN (
    'COMPANIES',
    'FISCAL_PERIODS',
    'INCOME_STATEMENTS',
    'BALANCE_SHEETS',
    'CASH_FLOW_STATEMENTS',
    'SEGMENTS',
    'SEGMENT_RESULTS',
    'DATA_SOURCES'
)
ORDER BY table_name;
```

2. Test the Number of Records in Every Table

```
SELECT 'COMPANIES' AS table_name, COUNT(*) AS record_count FROM companies
UNION ALL
SELECT 'FISCAL_PERIODS', COUNT(*) FROM fiscal_periods UNION ALL
SELECT 'INCOME_STATEMENTS', COUNT(*) FROM income_statements UNION ALL
SELECT 'BALANCE_SHEETS', COUNT(*) FROM balance_sheets UNION ALL
SELECT 'CASH_FLOW_STATEMENTS', COUNT(*) FROM cash_flow_statements UNION ALL
SELECT 'SEGMENTS', COUNT(*) FROM segments UNION
ALL
SELECT 'SEGMENT_RESULTS', COUNT(*) FROM segment_results UNION ALL
SELECT 'DATA_SOURCES', COUNT(*) FROM data_sources;
```

3. Test Company Data

```
SELECT *
FROM companies;
```

4. Test Fiscal Periods

```
SELECT *
FROM fiscal_periods ORDER
BY fiscal_year;
```

5. Test Income Statement Data

```
SELECT *
FROM income_statements ORDER
BY period_id;
```

6. Test Balance Sheet Data

```
SELECT *
FROM balance_sheets ORDER
BY period_id;
```

7. Test Cash Flow Data

```
SELECT *
FROM cash_flow_statements ORDER
BY period_id;
```

8. Test Segments

```
SELECT *
FROM segments
ORDER BY segment_id;
```

9. Test Segment Results

```
SELECT *
FROM segment_results
ORDER BY period_id, segment_id;
```

10. Test Source Records

```
SELECT *
FROM data_sources ORDER
BY source_id;
```

11. Test Total Revenue Calculation

```
SELECT period_id,
       net_sales,
       membership_other_income,
       total_revenues,
       net_sales + membership_other_income AS calculated_revenue, net_sales +
       membership_other_income - total_revenues AS difference
FROM income_statements ORDER
BY period_id;
```

12. Test Operating Income Calculation

```
SELECT period_id,
       total_revenues,
       cost_of_sales,
       operating_sga_expenses,
       operating_income,
       total_revenues - cost_of_sales - operating_sga_expenses AS calculated_operating_income, total_revenues - cost_of_sales -
       operating_sga_expenses - operating_income AS difference
FROM income_statements ORDER
BY period_id;
```

13. Test Ending Cash Calculation

```
SELECT period_id,
       beginning_cash,
       net_change_in_cash,
       ending_cash,
       beginning_cash + net_change_in_cash AS calculated_ending_cash, beginning_cash +
       net_change_in_cash - ending_cash AS difference
FROM cash_flow_statements ORDER
BY period_id;
```

14. Test Free Cash Flow

```
SELECT period_id,
       operating_cash_flow,
       capital_expenditures,
       operating_cash_flow - capital_expenditures AS free_cash_flow FROM cash_flow_statements
ORDER BY period_id;
```

15. Test Income Statement to Fiscal Period Relationship

```
SELECT
       fp.fiscal_year,
       i.total_revenues,
       i.operating_income,
       i.net_income_attributable_walmart FROM
fiscal_periods fp
JOIN income_statements i
  ON fp.period_id = i.period_id ORDER BY
fp.fiscal_year;
```

16. Test Balance Sheet to Fiscal Period Relationship

```
SELECT
       fp.fiscal_year,
       b.total_assets,
       b.total_current_assets,
       b.total_current_liabilities FROM
fiscal_periods fp
```

```
JOIN balance_sheets b
  ON fp.period_id = b.period_id ORDER BY
fp.fiscal_year;
```

17. Test Cash Flow to Fiscal Period Relationship

```
SELECT
  fp.fiscal_year,
  cf.operating_cash_flow,
  cf.capital_expenditures, cf.ending_cash
FROM fiscal_periods fp
JOIN cash_flow_statements cf
  ON fp.period_id = cf.period_id ORDER BY
fp.fiscal_year;
```

18. Test Segment JOIN

```
SELECT
  fp.fiscal_year,
  s.segment_name,
  sr.net_sales,
  sr.operating_income,
  sr.total_assets
FROM segment_results sr JOIN
fiscal_periods fp
  ON sr.period_id = fp.period_id JOIN segments s
  ON sr.segment_id = s.segment_id ORDER BY
fp.fiscal_year, s.segment_id;
```

19. Test Whether Segment Sales Equal Consolidated Net Sales

```
SELECT
  sr.period_id,
  SUM(sr.net_sales) AS total_segment_sales, i.net_sales AS
consolidated_net_sales,
  SUM(sr.net_sales) - i.net_sales AS difference FROM segment_results
sr
JOIN income_statements i
  ON sr.period_id = i.period_id GROUP BY
sr.period_id, i.net_sales ORDER BY sr.period_id;
```

20. Test Total Equity

```
SELECT period_id,
  walmart_shareholders_equity,
  noncontrolling_interest, total_equity,
  walmart_shareholders_equity + noncontrolling_interest AS calculated_equity, walmart_shareholders_equity +
  noncontrolling_interest - total_equity AS difference
FROM balance_sheets ORDER
BY period_id;
```

21. Check for Duplicate Fiscal Years

```
SELECT fiscal_year,
  COUNT(*) AS record_count FROM
fiscal_periods
GROUP BY fiscal_year
HAVING COUNT(*) > 1;
```

22. Check for Duplicate Segment/Year Combinations

```
SELECT period_id,
  segment_id,
  COUNT(*) AS record_count FROM
segment_results
GROUP BY period_id, segment_id HAVING
COUNT(*) > 1;
```

23. Check for Missing Income Statement Period IDs

```
SELECT *
FROM income_statements WHERE
period_id IS NULL;
```

24. Check for Missing Balance Sheet Period IDs

```
SELECT *
FROM balance_sheets
WHERE period_id IS NULL;
```

25. Check for Missing Cash Flow Period IDs

```
SELECT *
FROM cash_flow_statements WHERE
period_id IS NULL;
```

26. Check for Missing Segment IDs

```
SELECT *
FROM segment_results
WHERE segment_id IS NULL;
```

27. Check Source Verification

```
SELECT source_id,
period_id,
statement_name, verified
FROM data_sources WHERE
verified <> 'Y'
OR verified IS NULL;
```

28. Check Missing Source Information

```
SELECT *
FROM data_sources
WHERE filing_name IS NULL
OR source_section IS NULL OR
reporting_unit IS NULL;
```

29. Test Revenue Growth

```
SELECT fiscal_year,
total_revenues,
prior_year_revenue, ROUND(
(total_revenues - prior_year_revenue)
/ NULLIF(prior_year_revenue, 0) * 100,
2
) AS revenue_growth_pct FROM (
SELECT
fp.fiscal_year,
i.total_revenues,
LAG(i.total_revenues) OVER (ORDER BY fp.fiscal_year) AS prior_year_revenue FROM fiscal_periods fp
JOIN income_statements i
ON fp.period_id = i.period_id
)
ORDER BY fiscal_year;
```

30. Test Operating Margin

```
SELECT
fp.fiscal_year,
i.total_revenues,
i.operating_income, ROUND(
i.operating_income / NULLIF(i.total_revenues, 0) * 100,
2
) AS operating_margin_pct FROM
fiscal_periods fp
JOIN income_statements i
ON fp.period_id = i.period_id ORDER BY
fp.fiscal_year;
```

31. Test Net Profit Margin

```
SELECT
    fp.fiscal_year, ROUND(
        i.net_income_attributable_walmart / NULLIF(i.total_revenues, 0) * 100,
        2
    ) AS net_profit_margin_pct FROM
    fiscal_periods fp
JOIN income_statements i
    ON fp.period_id = i.period_id ORDER BY
    fp.fiscal_year;
```

32. Test Current Ratio

```
SELECT
    fp.fiscal_year,
    b.total_current_assets,
    b.total_current_liabilities, ROUND(
        b.total_current_assets / NULLIF(b.total_current_liabilities, 0),
        2
    ) AS current_ratio FROM
    fiscal_periods fp JOIN
    balance_sheets b
    ON fp.period_id = b.period_id ORDER BY
    fp.fiscal_year;
```

33. Test Segment Operating Margins

```
SELECT
    fp.fiscal_year,
    s.segment_name,
    sr.net_sales,
    sr.operating_income, ROUND(
        sr.operating_income / NULLIF(sr.net_sales, 0) * 100,
        2
    ) AS operating_margin_pct FROM
    segment_results sr
JOIN fiscal_periods fp
    ON sr.period_id = fp.period_id JOIN segments s
    ON sr.segment_id = s.segment_id ORDER BY
    fp.fiscal_year, s.segment_id;
```

34. Test Financial Summary View

```
SELECT *
FROM vw_walmart_financial_summary ORDER
BY fiscal_year;
```

35. Test Financial Ratio View

```
SELECT *
FROM vw_walmart_ratios ORDER
BY fiscal_year;
```

36. Test Segment View

```
SELECT *
FROM vw_walmart_segment_results ORDER BY
fiscal_year, segment_id;
```

37. Confirm the Views Exist

```
SELECT view_name
FROM user_views
WHERE view_name IN (
    'VW_WALMART_FINANCIAL_SUMMARY',
    'VW_WALMART_RATIOS',
    'VW_WALMART_SEGMENT_RESULTS'
)
ORDER BY view_name;
```